LECTURE 16

Error Handling and Fault Tolerance

Backend Engineering from First Principles

Detailed notes • architecture • examples • interview revision

1. The fault-tolerant mindset

Errors are normal in backend systems: database queries fail, external APIs time out, users submit invalid data, and business logic encounters edge cases. The question is not whether failures happen, but whether the system can detect, contain, and recover from them safely.

A fault-tolerant mindset means expecting failures, detecting them early, protecting data integrity, limiting their blast radius, and returning safe, useful responses.

2. Common categories of backend errors

2.1 Logic errors

A logic error occurs when code runs but produces an incorrect result. It may not crash the application, making it difficult to notice.

- Example from the lecture: an e-commerce platform applies a discount twice and creates negative shipping costs. Orders still complete, but the business loses money.
- Causes: misunderstood requirements, incorrect algorithms/calculations, missing edge cases, unexpected user behaviour, and insufficient testing.
- Prevention: clarify business rules, test boundaries and unusual flows, and monitor business outcomes—not just crashes.

2.2 Database errors

Backend systems rely heavily on databases, so database failures can affect much of the application.

- **Connection errors:** network problems, overloaded database, or exhausted connection pools. Pools reuse connections and reduce repeated connection-establishment overhead.
- **Constraint violations:** unique (duplicate email), foreign key (order references a nonexistent customer), and not-null violations.
- **Query errors:** malformed SQL, misspelled table/column names, missing database objects, or queries that time out.
- **Deadlocks/transaction issues:** concurrent transactions wait on resources held by one another in a cycle; the database may abort one transaction to break the deadlock.

Validation prevents many bad inputs, but database constraints remain the final authority—especially for uniqueness under concurrent requests. Convert known constraint errors into safe client responses.

2.3 External-service errors

Applications depend on payment providers, email services, object storage, authentication providers, and AI APIs. Each dependency is a potential failure point.

- Network failures: timeouts, DNS failures, network partitions.
- Authentication failures: bad credentials, expired tokens, insufficient permissions.
- Rate limits: HTTP **429 Too Many Requests**.
- Outages and maintenance.

Exponential backoff: increase the delay between retries after repeated transient failures (for example, 1, 2, 4, 8 time units). In production, respect `Retry-After` when available, add jitter, set a

retry budget, and do not retry permanent failures. Avoid retry storms that add load to an already struggling service.

Fallbacks: a backup node or in-memory cache may help for suitable functionality. Stale data may be acceptable for a non-critical feature, but not necessarily for payments or order processing.

2.4 Input-validation errors

Validation is the first line of defence at the backend entry point.

- Format: email, phone, date, or expected representation.
- Range/length: numeric limits, string length, array item count.
- Required fields: mandatory values must be present.

Invalid requests commonly receive **400 Bad Request**. Backend validation is required even when frontend validation exists.

2.5 Configuration errors

These often appear when moving from development to staging or production. A key may exist in a local `.env` file but be missing from the deployed environment.

Fail fast at startup: load and validate required configuration before accepting traffic. If a required value is missing or invalid, stop startup with a clear operator-facing error. This is preferable to starting successfully and returning runtime 500s only when a user invokes the affected feature.

3. Proactive error detection

The lecture's central principle: **the best error handling starts before an error happens.**

Health checks

A `/health` or `/status` endpoint lets infrastructure check whether a service responds. A simple 200 response only proves that the process is responding; it does not prove that it can perform its work.

- **Liveness:** is the process alive, or should it be restarted?
- **Readiness:** is this instance ready to receive traffic?

Dependency and core-functionality checks

Check more than the web server:

- Database connectivity and representative query performance.
- Payment-provider test transactions or approved health mechanisms.
- Email delivery tests to internal addresses.
- Authentication test-token validation.
- Required configuration, essential caches, and internal state initialization.

Avoid health checks that create real customer side effects.

4. Monitoring, observability, and logging

Monitoring detects problems quickly and provides debugging context. Do not track only error rates: performance degradation can be an early warning.

Area	Signals
HTTP/API	4xx/5xx rates, latency, throughput
Database	connection failures, pool saturation, query latency, deadlocks
External services	timeouts, provider errors, availability
Infrastructure	CPU, memory, disk, network, saturation
Business outcomes	successful payments, orders, authentications
Application logic	failed state transitions, unexpected business outcomes

A sudden drop in successful transactions can signal an issue even if HTTP error rates look normal.

Structured logging: JSON logs can be parsed, searched, enriched with metadata, and visualized. The lecture mentions Grafana and Loki. Useful fields include timestamp, severity, service, environment, request/correlation ID, operation, and safe error code. Never log passwords, API keys, access tokens, card data, or unnecessary sensitive information. Prefer an internal user ID over an email address.

5. Responding to errors

Recoverable errors

Temporary network errors, transient provider failures, and temporary resource exhaustion may be recoverable. Use bounded retries, exponential backoff, and queues where appropriate. Retries must not overwhelm an already stressed system.

Non-recoverable or unsafe-to-retry errors

Use **containment and graceful degradation**:

- Serve cached/last-known data only where safe.
- Disable non-essential features temporarily.
- Offer an alternative workflow.
- Switch to a safe fallback.
- Limit the failure to the affected component.

Do not silently use stale data for critical financial or transactional state.

6. Recovery and data integrity

Possible automated recovery actions include restarting an unhealthy process, rebuilding a corrupted cache, or switching to a backup. Test these mechanisms: an automatic restart or failover can worsen an incident if it repeatedly targets an unhealthy dependency.

Some incidents require human judgement. Document runbooks covering detection, escalation, safe actions, rollback/failover, and verification.

Data integrity is a top priority. Recovery practices include regular backups, tested restoration, restoring from a known-good backup, replaying transaction logs where supported, and database-specific recovery tools. An untested backup is not a proven recovery plan.

7. Error propagation and isolation

Typical layered flow:

`Route → Handler → Service → Repository → Database`

- **Repository:** focused persistence operation; may encounter database errors.
- **Service:** orchestrates business operations and adds business context.
- **Handler:** HTTP input, parsing, and validation.
- **Global error handler:** maps known failures to safe HTTP responses and handles unexpected errors.

In exception-based languages, errors may be thrown and caught at a higher boundary. In Go-style code, functions return errors that callers inspect and propagate. Preserve the underlying cause, add useful context, and avoid swallowing errors.

Error boundaries and isolation: separate processes/services where appropriate, use timeouts, and decouple work with message queues such as RabbitMQ. Asynchronous communication can prevent a failure in one component from immediately taking down another.

8. Global error handling: the final safety net

A global handler centralizes classification of internal errors and their mapping to client responses.

Example: create-book request

1. Route selects the handler.
2. Handler parses and validates input.
3. Service coordinates the operation.
4. Repository runs the database insert.
5. Errors propagate to the global handler.
6. The global handler returns a consistent, safe response.

Internal condition	Response in the lecture's example	Meaning
Book name exceeds allowed length	400	Invalid request data
Duplicate book name / unique constraint	400	Conflicting user data
Requested book has no matching row	404	Resource not found
Foreign-key violation: author ID absent	404	Referenced resource not found
Unknown internal failure	500	Generic server error

HTTP nuance: the lecture uses 400 for a duplicate book. `409 Conflict` is also commonly appropriate for uniqueness conflicts; choose and document a consistent API contract. A database “no rows” result is not automatically an error in every query—it becomes 404 when endpoint semantics require the resource to exist.

A consistent error response might look like:

```
{
"error": {
"code": "VALIDATION_ERROR",
"message": "Please correct the highlighted fields.",
"details": [
```

```
{"field": "name", "message": "Name must be 500 characters or fewer."}
],
"requestId": "req_..."
}
}
```

Why centralize?

- Consistent responses across layers.
- Fewer missed error cases and less duplicated mapping logic.
- Less risk of exposing raw database details.
- Repositories can focus on persistence rather than HTTP formatting.

9. Security practices for errors and logs

Do not leak internal details

Raw database errors may reveal table names, indexes, constraint names, paths, or implementation details. Map known failures to safe messages. For unexpected 500s, return a generic message such as “Something went wrong.” Keep diagnostic details in protected internal logs and provide a request ID for investigation.

Prevent account enumeration

A login endpoint should not distinguish “email does not exist” from “password is incorrect.” Attackers can use detailed differences to discover registered accounts and then target them.

Use a generic message such as **“Invalid email or password.”** Keep response behaviour as consistent as practical and apply rate limiting.

Protect logs

Logs may pass through external storage and observability providers. Do not log passwords, API keys, access tokens, credit-card data, or unnecessary sensitive personal information. Prefer internal user IDs and correlation IDs over emails. Restrict access and apply retention/redaction policies.

10. Practical checklist

Before deployment

- ■ Clarify requirements and edge cases.
- ■ Validate incoming data on the backend.
- ■ Test critical calculations and state transitions.
- ■ Validate required configuration before startup.
- ■ Test database constraints and transaction behaviour.
- ■ Define consistent error categories and response formats.
- ■ Ensure unexpected errors produce safe 500 responses.

At runtime

- ■ Monitor health, readiness, latency, throughput, and resource usage.

- Track database and external-provider failures.
- Track business success metrics.
- Use bounded retries with backoff for transient failures.
- Avoid retrying permanent errors.
- Use timeouts and isolate dependency failures.
- Degrade gracefully only where semantically safe.
- Preserve request IDs and internal diagnostic context.

Security and recovery

- Do not expose stack traces or raw database errors.
- Use generic login errors to prevent account enumeration.
- Redact secrets and sensitive data from logs.
- Maintain tested backups and restore procedures.
- Document and rehearse incident runbooks.
- Test failover and recovery mechanisms.

11. Interview questions and answers

1. **What is a logic error?** Code runs but produces an incorrect result; it may not crash.
2. **Why can constraint violations occur despite validation?** Concurrent requests can race; the database is the final authority.
3. **What is a connection pool?** A managed set of reusable DB connections that reduces repeated connection setup overhead.
4. **What is a deadlock?** A circular wait between concurrent transactions.
5. **What does 429 mean?** Too Many Requests; a rate limit was exceeded.
6. **What is exponential backoff?** Increasing retry delays after repeated failures, usually with limits and jitter.
7. **Why validate configuration at startup?** To catch deployment errors before users reach affected features.
8. **Is a basic health endpoint enough?** No; readiness and dependency checks are also important.
9. **What is graceful degradation?** Keeping essential functionality available while reducing non-essential functionality.
10. **Why use a global error handler?** Consistency, less duplication, and safer client responses.
11. **What is error propagation?** Passing an error to a higher layer with more context to decide how to handle it.
12. **Why generic login errors?** To prevent account enumeration.
13. **What should not be logged?** Passwords, tokens, API keys, card data, and unnecessary sensitive information.
14. **Why monitor business metrics?** Technical error rates may look normal while successful transactions fall.

15. **How do message queues help?** They decouple services and allow asynchronous work.

12. Quick revision sheet

- Logic error: code runs, result is wrong.
- Database: connection, constraint, query, deadlock/transaction.
- External dependency: network, auth, rate limit (429), outage.
- Validation: format, range/length, required fields; commonly 400.
- Configuration: validate required values before startup; fail fast.
- Health: process health alone is insufficient; check readiness and dependencies.
- Monitoring: errors + latency + resource usage + throughput + business metrics.
- Recoverable: bounded retry, exponential backoff, jitter.
- Unsafe/non-recoverable: contain, degrade gracefully, safe fallback.
- Recovery: tested restart/failover, runbooks, backups, restore, transaction-log replay.
- Error flow: repository → service → handler → global boundary.
- Global handler: consistent mapping, less duplication, safe responses.
- Security: no internal leakage, no account enumeration, no secrets in logs.

Based on the supplied lecture transcript. Additional engineering nuance is explicitly identified where it goes beyond the speaker's examples.